

Lessons learned from the early performance evaluation of Intel Optane DC Persistent Memory in DBMS

Yinjun Wu¹, Kwanghyun Park², Rathijit Sen², Brian Kroth², Jaeyoung Do³

¹University of Pennsylvania, ²Microsoft Gray Systems Lab, ³Microsoft Research

ABSTRACT

Non-volatile memory (NVM) is an emerging technology, which has the persistence characteristics of large capacity storage devices, while providing the low access latency and byte-addressability of traditional DRAM memory. In this paper, we provide extensive performance evaluations on a recently released NVM device, Intel Optane DC Persistent Memory (PMem), under different configurations with several micro-benchmark tools. Further, we evaluate OLTP and OLAP database workloads with Microsoft SQL Server 2019 when using PMem as buffer pool or persistent storage. From the lessons learned we share some recommendations for future DBMS design with PMem, e.g. simple hardware or software changes are not enough for the best use of PMem in DBMSs.

1 INTRODUCTION

Prior to the release of Intel Optane DC persistent memory (PMem for short hereafter), a series of research works have focused on the use of PMem in DBMS with the NVM emulated in DRAM (e.g., see [8, 10, 11, 16, 18]). Therefore, we anticipate that the recently released PMem will motivate database researchers to revisit those designs and explore new solutions for PMem-aware DBMS [7].

The focus of this paper is to explore PMem's characteristics, mainly in the DBMS perspective, under several PMem configurations with micro-benchmark tools. The observations not only complement the existing performance characterizations on this device [4–6, 14, 15, 19, 21], but also help explain database workload performance results and inform our suggestions for optimal DBMS internal parameters for query processing on PMem. In addition, we evaluate OLTP and OLAP workloads (i.e., TPC-C and TPC-H) with different PMem configurations to explore the potential database design choices when integrating PMem in DBMSs. The evaluation results point out that the solutions so far are suboptimal. Therefore there are huge opportunities for additional research to properly adopt PMem for DBMS internals.

2 PMEM MICRO-BENCHMARK ANALYSIS

This section presents performance analysis of PMem device characteristics in the following three different modes: 1) *Memory mode*: PMem becomes the *volatile* main memory with DRAM as a L4 cache; 2) *DAX mode*: applications can issue direct CPU data access

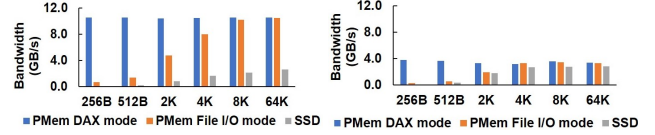
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAMON'20, June 15, 2020, Portland, OR, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8024-9/20/06...\$15.00

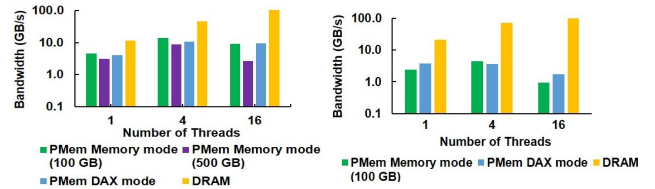
<https://doi.org/10.1145/3399666.3399898>



(a) Read

(b) Write

Figure 1: Sequential-access peak bandwidth results



(a) Read

(b) Write

Figure 2: Sequential read/write bandwidth results when PMem is used as memory.

to PMem via load and store instructions; 3) *File I/O mode*: PMem is accessed by applications using standard file system APIs.

2.1 System configuration

We conduct all experiments on a dual-socket server with Intel Xeon Platinum 8260L CPUs. Each socket has 24 physical cores, and is also populated with six 16GB Micron DDR4, and two interleaved 512GB Intel Optane DC Persistent Memory modules. We run Ubuntu 18.10 on the server where hyper-threading is enabled and the CPU power governor is configured to performance mode.

Note that all experiments in this section are performed in one socket accessed by the local CPUs. To compare performance when PMem is used as persistent storage, we use one NVMe 4TB Intel DC P4510 Series SSD (called SSD for short hereafter).

2.2 Microbenchmark results

First of all, we use Flexible I/O tester (FIO) [1] to characterize the performance of PMem when it is used as persistent storage in either DAX or File I/O mode, compared against SSD. In addition, we use Memory Latency checker (MLC) [3] to characterize the performance of PMem memory mode with two different data sizes (100GB and 500GB), which is compared with DRAM and PMem DAX mode. Due to space limits, for FIO results, we only present the peak bandwidth results of sequential access with varied I/O request sizes in Figure 1¹ while for MLC results, we present the bandwidth of sequential access with varied thread count in Figure 2². In the

¹Note that the minimum I/O size for SSD is limited to its device sector size (i.e., 512B) while there is no such limitation for PMem in DAX and File I/O modes.

²Note that during the write performance measurements on 500GB dataset in PMem Memory Mode, MLC crashed due to some internal issues.

end, we have the following observations relevant to the database workload evaluation analysis (more details can be found in [20]):

- (1) *The performance of reads and writes of the PMem DAX mode and PMem File I/O mode are asymmetric. In contrast, SSD provides more balanced performance between reads and writes.*
- (2) *With larger data size or under highly parallel write workloads with smaller data size, PMem Memory Mode performs worse than PMem DAX.*

3 DATABASE WORKLOAD EVALUATIONS

We will now present our performance evaluation of typical OLTP and OLAP workloads, specifically, TPC-C and TPC-H workloads, with different PMem configurations in Microsoft SQL Server 2019.

3.1 Experimental design

By limiting the CPU access within one socket and using the same tempdb directory throughout the experiments, we compare the performance of the following configurations in SQL Server: 1) *PMem DAX* which uses the Hybrid buffer pool, whereby the warm pages cached in the extended buffer pool of the PMem are accessed with DAX; 2) *PMem File I/O* which issues File I/O against PMem when it is configured as persistent storage of SQL Server; 3) *SSD + DRAM* is the traditional DBMS configuration with SSD as the persistent storage and DRAM as the buffer pool; 4) *SSD + PMem Memory Mode* uses PMem in Memory Mode as the buffer pool in SQL Server with DRAM as an L4 cache and SSD as the persistent storage.

We study two typical database workloads — TPC-C and TPC-H. For the TPC-C experiments, we use OLTPBench [12, 13] to issue the queries and configure each run to last 30 minutes. We also set the checkpointing recovery interval in SQL Server to 60 minutes to disable the effect of checkpoints. For the TPC-H experiments, we warm up the buffer pool by running each query for multiple times and only consider the runtime for the last execution. We use two different scale factors — SF 100 and SF 500 for TPC-H and SF 1300 and SF 6500 for TPC-C. These generate database instances with sizes of approximately 100GB and 500GB respectively.

3.2 TPC-H benchmark

Due to space constraints, we present a subset of the performance results for TPC-H queries in Figures 3 with the following analysis:

Performance of PMem DAX With smaller scale factor, i.e., SF=100, PMem DAX fails to outperform SSD + DRAM and PMem File I/O (~25s vs. ~5s for Query 3). As explained in [2], the use of the Hybrid buffer pool with DAX is to reference the pages in PMem instead of caching them in DRAM buffer pool, thus failing to utilize the benefit of the DRAM buffer pool. In contrast, for SF=100, by using *PMem File I/O* and *SSD + DRAM* large portions of the hot pages are cached in DRAM buffer pool after their first access, leading to smaller overhead in subsequent page accesses and, thus, shorter query execution time. This is confirmed by more memory usage for *PMem File I/O* and *SSD + DRAM* than that for *PMem DAX* (~150GB vs. ~20GB). In case of larger SF, i.e. SF=500, only a small portion of the hot pages can be cached in the DRAM buffer pool for *PMem DAX*, *PMem File I/O* and *SSD + DRAM*, which means that the I/O overhead will become more prominent. This leads to shorter

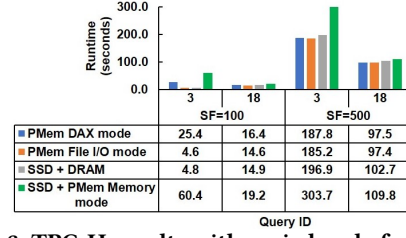


Figure 3: TPC-H results with varied scale factors (SF)

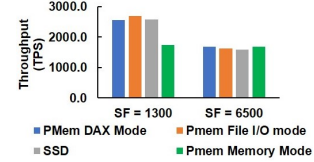


Figure 4: TPC-C results with varied scale factors (SF)

runtime on PMem DAX and PMem File I/O than that on SSD + DRAM (~187s vs. ~197s resp. for Query 3).

Recommendations This unexpected result indicates the need for further research into good PMem-aware page placement policies for the buffer pool in DBMS.

Performance of SSD + PMem Memory Mode The performance of this configuration is worse than that of SSD + DRAM. For example, with SF=100, the runtime of Query 3 with SSD + PMem Memory Mode is ~60s, ~12x slower than that of SSD + DRAM (~5s). This is caused by the writes of substantial intermediate results on memory during query executions. As explained in [5] and Section 2.2, write operations on PMem Memory Mode can lead to performance loss, thus longer query processing time.

Recommendations This result indicates that it is not appropriate to directly replace DRAM with PMem in Memory Mode as the buffer pool in DBMS even for (read intensive) TPC-H workloads because of the writes to tempdb for intermediate query results.

3.3 TPC-C benchmark

We show TPC-C results in Figure 4, which shows that there is no significant performance difference between PMem File I/O, PMem DAX, and SSD + DRAM under TPC-C workloads. This is because the read-write performance asymmetry of PMem as mentioned in Section 2.2 leads to the performance loss in TPC-C workload which is a write-intensive workload.

Recommendations The downside of the write operations on PMem has motivated some early works (e.g. [9, 17, 18]) on limiting write operations before the appearance of the real PMem device. It would be valuable to revisit the feasibility of those ideas on the real PMem device.

4 CONCLUSION AND FUTURE WORK

In this paper, we explored some device characteristics of PMem, followed by database workload evaluations on DBMSs with the use of PMem, which reveal that simple hardware or software changes will not be properly integrating PMem in DBMSs. For the best use of PMem, many DBMS internals should be revisited in the future.

REFERENCES

- [1] Flexible I/O tester (FIO) rev. 3.15. https://fio.readthedocs.io/en/latest/fio_doc.html.
- [2] Hybrid buffer pool feature in SQL server. <https://docs.microsoft.com/en-us/sql/database-engine/configure-windows/hybrid-buffer-pool?view=sql-server-ver15>.
- [3] Intel Memory Latency Checker v3.8. <https://software.intel.com/en-us/articles/intelr-memory-latency-checker>.
- [4] Analyzing the Performance of Intel Optane DC Persistent Memory in App Direct Mode in Lenovo ThinkSystem Servers. <https://lenovopress.com/lp1083.pdf>, 2019.
- [5] Analyzing the Performance of Intel Optane DC Persistent Memory in Memory Mode in Lenovo ThinkSystem Servers. <https://lenovopress.com/lp1084.pdf>, 2019.
- [6] HPE Persistent Memory performance in HPE ProLiant, HPE Synergy, and HPE Apollo Gen10 servers with second-generation Intel Xeon Scalable processors. <https://h20195.www2.hpe.com/v2/getdocument.aspx?docname=a00075993enw>, 2019.
- [7] Joy Arulraj and Andrew Pavlo. Non-volatile memory database management systems. *Synthesis Lectures on Data Management*, 11(1):1–191, 2019.
- [8] Joy Arulraj, Matthew Perron, and Andrew Pavlo. Write-behind logging. *Proceedings of the VLDB Endowment*, 10(4):337–348, 2016.
- [9] Daniel Bausch, Ilia Petrov, and Alejandro Buchmann. Making cost-based query optimization asymmetry-aware. In *Proceedings of the Eighth International Workshop on Data Management on New Hardware*, pages 24–32, 2012.
- [10] Guy E Blelloch, Jeremy T Fineman, Phillip B Gibbons, Yan Gu, and Julian Shun. Sorting with asymmetric read and write costs. In *Proceedings of the 27th ACM symposium on Parallelism in Algorithms and Architectures*, pages 1–12. ACM, 2015.
- [11] Shimin Chen and Qin Jin. Persistent b+-trees in non-volatile main memory. *Proceedings of the VLDB Endowment*, 8(7):786–797, 2015.
- [12] Carlo A Curino, Djellel E Difallah, Andrew Pavlo, and Philippe Cudre-Mauroux. Benchmarking oltp/web databases in the cloud: The oltp-bench framework. In *Proceedings of the fourth international workshop on Cloud data management*, pages 17–20, 2012.
- [13] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. Oltp-bench: An extensible testbed for benchmarking relational databases. *Proceedings of the VLDB Endowment*, 7(4):277–288, 2013.
- [14] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amir Saman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R Dullloor, et al. Basic performance measurements of the intel optane dc persistent memory module. *arXiv preprint arXiv:1903.05714*, 2019.
- [15] Ivy B Peng, Maya B Gokhale, and Eric W Green. System evaluation of the intel optane byte-addressable nvm. In *Proceedings of the International Symposium on Memory Systems*, pages 304–315, 2019.
- [16] Alexander van Renen, Viktor Leis, Alfons Kemper, Thomas Neumann, Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi, Lilian Harada, and Mitsuru Sato. Managing non-volatile memory in database systems. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1541–1555. ACM, 2018.
- [17] Stratis D Viglas. Adapting the b+-tree for asymmetric i/o. In *East European Conference on Advances in Databases and Information Systems*, pages 399–412. Springer, 2012.
- [18] Stratis D Viglas. Write-limited sorts and joins for persistent memory. *Proceedings of the VLDB Endowment*, 7(5):413–424, 2014.
- [19] Michèle Weiland, Holger Brunst, Tiago Quintino, Nick Johnson, Olivier Iffrig, Simon Smart, Christian Herold, Antonino Bonanni, Adrian Jackson, and Mark Parsons. An early evaluation of intel's optane dc persistent memory module and its impact on high-performance scientific applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–19, 2019.
- [20] Yinjun Wu, Kwanghyun Park, Rathijit Sen, Brian Kroth, and Jaeyoung Do. Lessons learned from the early performance evaluation of intel optane dc persistent memory in DBMS. *arXiv preprint arXiv:2005.07658*, 2020. URL: <https://arxiv.org/pdf/2005.07658.pdf>.
- [21] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steven Swanson. An empirical guide to the behavior and use of scalable persistent memory. *arXiv preprint arXiv:1908.03583*, 2019.